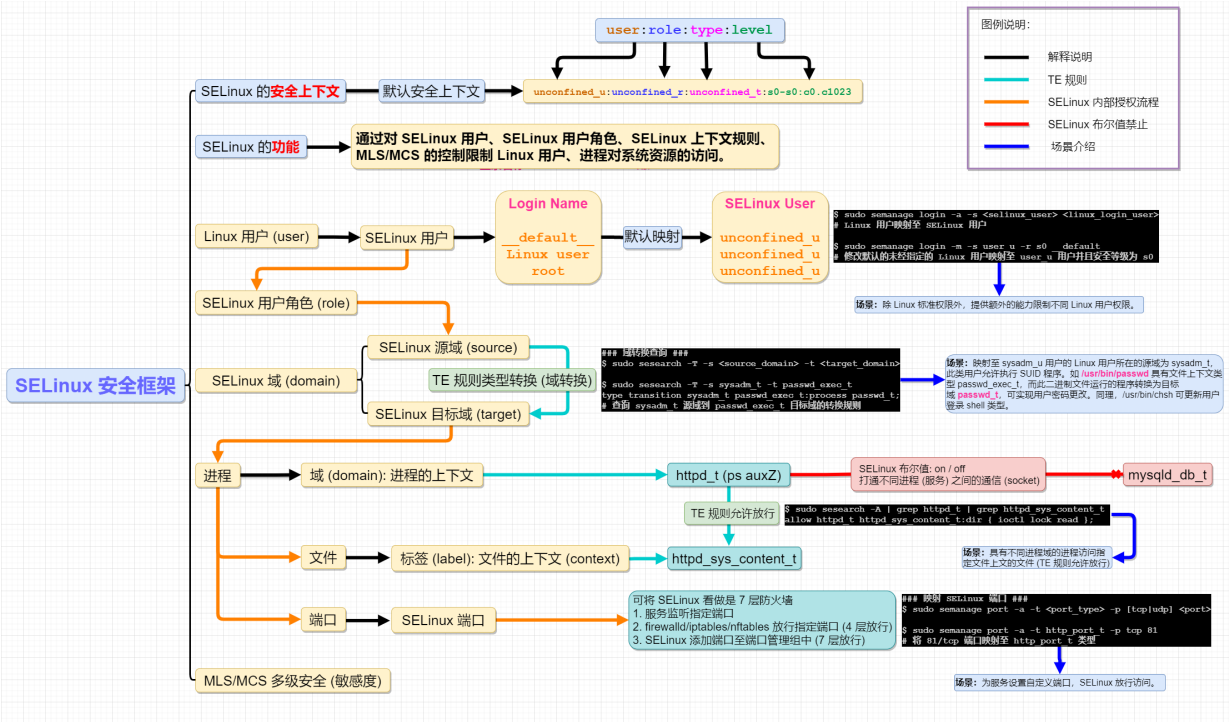
🧱 快速上手 SELinux 强化 Linux 安全性

文档目录

- 🧱 快速上手 SELinux 强化 Linux 安全性
  - 文档目录
  - 0. SELinux 安全框架
  - 1. 从禁用状态启用 SELinux
    - 1.1 复习 SELinux 基本概念
    - 1.2 配置 SELinux 模式
    - 1.3 从禁用模式启用 SELinux
  - 2. 使用受限用户控制访问
    - 2.1 定义 SELinux 用户（SELinux User）
    - 2.2 🎯 文件上下文类型与源域、目标域的关系
      - 2.2.1 关系总览
      - 2.2.2 文件上下文类型作为桥梁
      - 2.2.3 文件上下文类型作为入口点（Entrypoint）
      - 2.2.4 完整 Domain Transition 需要三条规则
      - 2.2.5 示例：sshd 启动完整流程
    - 2.3 限制（confine）用户账户
  - 3. 参考链接

0. SELinux 安全框架



1. 从禁用状态启用 SELinux

1.1 复习 SELinux 基本概念

- **SELinux 核心功能：SELinux（Security-Enhanced Linux, 安全增强的 Linux）通过强制访问控制（Mandatory Access Control, MAC）机制，结合类型强制（TE）、基于角色的访问控制（RBAC）和多级安全（MLS/MCS），对 Linux 用户、进程及系统资源实施细粒度的安全隔离与访问限制。MAC 机制由内核实现，对用户透明，用户在启用 SELinux 的状态下无法绕过。**
- SELinux 通过以下核心机制实现强制访问控制：
  - 身份标识：SELinux 用户（User）与 Linux 用户映射
  - 权限边界：角色（Role）约束用户可访问的域
  - 类型强制：安全上下文（Context）中的类型（Type）决定资源访问规则
  - 层级隔离：MLS/MCS 实现数据机密性和类别隔离
- **最终效果：限制进程只能访问其被明确允许的资源，防止权限提升和横向移动，即使账户被攻破也能最小化损害。**

> 从层级关系理解 SELinux 实现机制： SELinux 是一种强制访问控制（MAC）安全机制，通过以下层次化策略
> 限制 Linux 主体（用户、进程）对客体（文件、端口、资源）的访问：
>
> 1. 身份层：Linux 用户 → 映射到 SELinux User（标识安全身份）
> 2. 角色层：SELinux User → 授权访问特定 Role（权限边界）
> 3. 类型层：Role → 允许进入特定 Domain（Type），Domain 通过 TE（Type Enforcement，类型强制）规则决定可访问的 Resource Type（**核心强制机制**）
> 4. 层级层：MLS/MCS 提供额外的敏感度/类别隔离（可选高级特性）
>
> **最终安全上下文格式：** `user:role:type:level`

- MAC 机制与 DAC（discretionary /dɪˈskreʃənəri/ adj. 自由决定的）机制对比：

对比项	DAC（自主访问控制）	MAC（强制访问控制）
控制者	资源所有者（用户）	系统安全策略
权限修改	用户可自由更改	用户无法更改
粒度	粗（用户/组/其他）	细（标签、类型、级别）
安全性	低（依赖用户自觉）	高（强制隔离）
典型系统	传统 Linux/Unix 权限	SELinux、AppArmor、Trusted Solaris

- SELinux 可以充分限制 **程序（主要是系统服务）** 的权限，仅授予程序成功完成工作所需的权限。
- SELinux 有一组安全规则确定哪个进程可以访问哪些文件、目录和端口。
- 若没有明确的访问规则，SELinux 会拒绝操作。
- ☑ **每个文件、进程和端口均有一个特定的安全标签（security label），称为 SELinux 上下文（context）。**
- SELinux 规则使用上下文控制访问。
- ★ 文件和目录：SELinux 会将上下文存储为文件系统的扩展属性（xattr）。
- 进程和端口：内核会将上下文保留在内存中，进程的上下文类型也称为域（domain）。
- RHEL 中 SELinux 的大部分规则基于第三个元素，即 **上下文类型（context type）**。
- 更改文件和目录的 SELinux 上下文

- 定义 SELinux 默认文件上下文规则
- 标记 SELinux 端口
- 使用 SELinux 布尔值：
  - SELinux 布尔值是更改 SELinux 策略行为的开关。
  - 可以启用或禁用 SELinux 的访问规则集。
  - 安全管理员可以使用 SELinux 布尔值来有选择地调整策略。

> 🔍 注意：SELinux 文件上下文规则更改、SELinux 端口标记、切换 SELinux 布尔值等可参考此 脚本 的最后注释中的 Note 部分。

- 访问文档：
  - 每个 SELinux 命令都有一个 man page
  - 每个 semanage 子命令也都有自己的 man page
  - 域（进程的上下文类型）也具有 man page

```
$ sudo dnf install -y policycoreutils-devel
# 安装 SELinux 核心策略开发工具软件包

$ sudo sepolicy manpage -a -p /path/to/dir
# 指定路径为所有域构建 man page
# -a 选项：为所有域构建 man page
# -p 选项：指定路径构建 man page（标准的man page存放路径为 /usr/share/man/man8/），默认构建于 /tmp 目录中。

$ sudo sepolicy manpage -a -p /opt/selinux-man    # 指定目录创建所有域 man page
$ man --ascii /opt/selinux-man/sssd_selinux.8     # 解析 ASCII 格式显示 man page

$ sudo sepolicy manpage -d <domain>
# 为指定的域构建 man page

$ sudo sepolicy manpage -d sshd_t -p /opt
# 在 /opt 目录中为 sshd_t 域构建 man page
```

1.2 配置 SELinux 模式

- 若将现有系统直接从 disabled 模式切换到 enforcing 模式，有可能会发现访问违规和其他 SELinux 问题。
- 通常而言，最好是安装时将系统配置为 permissive 模式或 enforcing 模式。
- **系统由 permissive 模式转换至 enforcing 模式要比 disabled 模式转换更加容易。**

- RHEL9.x 中临时修改 SELinux 模式：

```
$ sudo getenforce
# 获取当前 SELinux 状态

$ sudo setenforce 0
# 临时修改 SELinux 状态为 Permissive，重启后失效。
$ sudo setenforce 1
# 临时修改 SELinux 状态为 Enforcing，重启后失效。
# 注意：无法在系统运行状态中禁用（Disabled）SELinux
```

- RHEL9.x 中永久修改 SELinux 模式：

```
$ sudo vim /etc/selinux/config
...
SELINUX=disabled
...
# 永久禁用 SELinux

$ sudo grubby --update-kernel ALL --args selinux=0
# 内核命令行参数永久禁用 SELinux
$ sudo grubby --update-kernel ALL --remove-args selinux
# 内核命令行参数永久启用 SELinux
```

1.3 从禁用模式启用 SELinux

- 将 SELinux 从 disabled 模式切换至 enforcing 模式下，需遵循如下步骤，以免 SELinux 阻止应用。
- 将系统切换到 permissive 模式。
- 在此模式下，SELinux 不会强制执行该策略，但至少会在 /var/log/audit/audit.log 文件中报告拒绝记录。
- disabled 模式切换到 permissive 模式，需重启系统并重新标记文件系统。
- 再次查看 audit.log 文件的拒绝记录。
- permissive 模式下 SELinux 不会强制拒绝。
- 解决拒绝，需要分析每个拒绝，掌握可疑应用的行为，然后才能做出适当的响应。
- 重复步骤将系统切换到 permissive 模式和解决拒绝，需要分析每个拒绝，掌握可疑应用的行为，然后才能做出适当的响应，直到所有拒绝均已解决为止。

- 在经过了确认已解决的所有问题后，切换到 enforcing 模式。
- 查看 SELinux 访问冲突审计事件：
  - SELinux 拒绝访问资源之后，还将记入一个审计事件。
  - 若 auditd 服务已启动，系统将会把 SELinux 审计事件记入 /var/log/audit/audit.log。
  - 否则，系统将 SELinux 消息发送至 /var/log/messages 文件。
  - 示例：创建自定义的 Apache httpd 根目录 /web/data，配置基于域名的虚拟主机后访问该站点，若不设置根目录的 SELinux 类型上下文，将拒绝访问。

```
$ sudo ausearch -i -m AVC -ts boot
# 查看自启动后相关的 SELinux 审计记录

$ sudo ausearch -i -m AVC,USER_AVC,SELINUX_ERR,USER_SELINUX_ERR -ts recent
# 查看最近 10 分钟的 SELinux 审计记录
```

- 使用允许域：
  - 在 permissive 模式下，SELinux 将记录它应该拒绝的访问，但始终都会允许访问。
  - 若将系统切换到 permissive 模式（可能为调试特定进程），此举可能会造成严重的安全后果，因为其他所有服务此时也将无法得到 SELinux 的保护。
  - 有些情况下，可能正在将系统从 disabled 模式切换到 enforcing 模式，而且只剩下一两个 SELinux 域的 AVC 问题还未解决。
  - 多数情况下，可以将系统设置为 enforcing 模式，但应当在确认相关问题之后，再将这一两个域由 permissive 模式切换到 enforcing 模式。
  - 可以将系统设为 enforcing 模式，只将特定域设为 permissive 模式。

```
$ sudo semanage permissive -a <domain>
# 设置特定域为 permissive 模式使其成为无限制域，但策略依然将记录访问的违规记录。

$ sudo semanage permissive -a httpd_t
# 将 Apache httpd_t 域设置为 permissive 模式

$ sudo semanage permissive -l
# 列出当前处于 permissive 模式的域

$ sudo semanage permissive -d httpd_t
# 将 Apache httpd_t 域切换为 enforcing 模式
```

- 使用 Ansible 修复 SElinux：

- 若具有一组相同的系统，运行相同的应用，则可使用 Ansible 配置 SELinux，然后在这些系统上以可重现且相同的方式修复拒绝。
- 在 Ansible 控制节点上安装 rhel-system-roles 软件包即可部署这些角色。
- SELinux 角色（role）的功能：
  - 设置 SELinux 的模式：enforcing、permissive、disabled
  - 恢复文件和目录的上下文（等同于 restorecon 命令）
  - 标记 SELinux 端口（等同于 semanage port 命令）
  - 添加新的文件上下文规则（等同于 semanage fcontext 命令）
  - 切换布尔值（等同于 setsebool 命令）
- SELinux 角色示例：确保 SELinux 处于 enforcing 模式，/virtual 目录的内容设置了类型 httpd_sys_content_t，端口 8080/TCP 的标签已设为 http_port_t，并且已启用了SELinux 的布尔值 httpd_enable_homedirs。

---

2. 使用受限用户控制访问

2.1 定义 SELinux 用户（SELinux User）

- SELinux 策略对其 SELinux 用户有所定义，与 Linux 用户截然不同。
- **Linux 用户登录时，被映射到一个 SELinux 用户。**
- 与 Audit UID 一样，SELinux 用户在登录会话期间无法更改。
- 通常，多个 Linux 用户可以映射到同一 SELinux 用户。
- 该策略能对具体某些 SELinux 用户可以执行的操作施加额外的 SELinux 强制限制。

```
$ sudo semanage user -l

                Labeling   MLS/       MLS/
SELinux User    Prefix     MCS Level  MCS Range                      SELinux Roles

guest_u         user       s0         s0                             guest_r
root            user       s0         s0-s0:c0.c1023                 staff_r sysadm_r
system_r unconfined_r
staff_u         user       s0         s0-s0:c0.c1023                 staff_r sysadm_r
system_r unconfined_r
sysadm_u        user       s0         s0-s0:c0.c1023                 sysadm_r
system_u        user       s0         s0-s0:c0.c1023                 system_r
unconfined_r
unconfined_u    user       s0         s0-s0:c0.c1023                 system_r
unconfined_r
user_u          user       s0         s0                             user_r
xguest_u        user       s0         s0                             xguest_r
# 查看各个 SELinux 用户与对应的一组 SELinux 角色
```

- 【第一个字段】SELinux 用户（SELinux User）的说明：

SELinux 用户	功能作用	典型用途
`unconfined_u`	**无限制用户**，拥有几乎完全的访问权限，不受 SELinux 策略的严格限制。默认情况下，在全新的 RHEL 安装中，Linux 用户将映射到 SELinux 用户 unconfined_u。	普通用户、管理员日常操作
`sysadm_u`	**系统管理员用户**，拥有管理系统的高权限，但受 SELinux 审计和策略约束。	系统维护、配置管理（需要进行系统配置但不希望完全无限制的管理员）
`user_u`	**受限普通用户**，权限受到严格限制，不能执行特权操作，不能执行 SUID 程序，不能 su 切换到其他用户。	普通终端用户、需要最小权限原则的环境
`guest_u`	**来宾用户**，权限极度受限，通常只能访问自己的文件，不能访问网络、不能执行大多数程序。	临时账户、公共访问终端
`staff_u`	**工作人员用户**，类似普通用户但可能有一些额外权限。	开发人员、技术支持
`system_u`	**系统用户**，用于运行系统服务和守护进程，不用于交互式登录。	系统服务（httpd、sshd 等）
`root`	**超级用户**，在 SELinux 下也需要映射到特定 SELinux user。	通常映射为 `unconfined_u` 或 `sysadm_u`
`xguest_u`	**极受限来宾用户**，通常用于 kiosk 模式或浏览器沙箱。	公共终端、浏览器沙箱
`dbadm_u`	**数据库管理员用户**，专门管理数据库服务。	DBA 角色
`webadm_u`	**Web 管理员用户**，专门管理 Web 服务。	Web 服务器管理

- 【第二个字段】SELinux 用户角色（Role）的说明：

SELinux Role	**功能作用**	**典型关联的** User	**可访问的** Domain **示例**
`unconfined_r`	**无限制角色**，几乎不受限制，可访问大多数域。	`unconfined_u`	`unconfined_t`
`sysadm_r`	**系统管理员角色**，可执行系统管理任务。	`sysadm_u`	`sysadm_t`，`init_t` 等
`user_r`	**普通用户角色**，受限访问，只能操作普通用户域。	`user_u`，`staff_u`	`user_t`，`staff_t`
`guest_r`	**来宾角色**，极度受限，只能访问基本资源。	`guest_u`	`guest_t`
`staff_r`	**工作人员角色**，介于 user 和 sysadm 之间。	`staff_u`	`staff_t`，可切换到 `sysadm_r`
`system_r`	**系统角色**，专用于系统进程和守护进程。	`system_u`	各种服务域（`httpd_t`，`sshd_t` 等）
`secadm_r`	**安全管理员角色**，专门管理 SELinux 策略。	`secadm_u`	`secadm_t`
`auditadm_r`	**审计管理员角色**，管理审计系统。	`auditadm_u`	`auditadm_t`
`dbadm_r`	**数据库管理员角色**，管理数据库服务。	`dbadm_u`	`dbadm_t`，`postgresql_t` 等
`webadm_r`	**Web 管理员角色**，管理 Web 服务。	`webadm_u`	`webadm_t`，`httpd_t`
`xguest_r`	**极受限来宾角色**，用于浏览器沙箱等。	`xguest_u`	`xguest_t`

- 将 Linux 用户映射到 SELinux 用户：
  - 登录时，SELinux 会将 Linux 用户映射到 SELinux 用户。
  - 因此，Linux 用户会继承相应 SELinux 用户受到的限制。

```
$ id -Z
unconfined_u:unconfined_r:unconfined_t:s0-s0:c0.c1023
# 查看登录的 Linux 用户关联的 SELinux 用户（第一个字段）

$ sudo semanage login -l

Login Name           SELinux User         MLS/MCS Range        Service

__default__          unconfined_u         s0-s0:c0.c1023       *
root                 unconfined_u         s0-s0:c0.c1023       *
# 查看默认 Linux 用户映射到 SELinux 用户的关系
# SELinux 将 Linux root 用户映射到 unconfined_u SELinux 用户。
# __default__ 条目：SELinux 将未明确映射到 SELinux 用户的所有 Linux 用户映射到
unconfined_u SELinux 用户。
```

修改默认映射

示例1

$ sudo semanage login -m -s user_u -r s0 **default**

修改系统默认的 default 映射，使得所有未指定明确映射的 Linux 用户，

在登录时默认映射到 user_u SELinux 用户，并分配 s0 级别。

$ sudo semanage login -l

Login Name SELinux User MLS/MCS Range Service

default user_u s0 * # 已修改 root unconfined_u s0-s0:c0.c1023 *

示例2

sudo semanage login -a -s system_u seuser1 # 必须先映射 Linux 用户至 SELinux 用户 sudo semanage login -m -s user_u -r s0 seuser1 # 再修改映射关系

> 注意：必须重新使用 SSH 登录的方式验证 Linux 用户与 SELinux 用户的映射关系，不可使用 su 命令进行用户切换。

```bash
$ sudo useradd -Z <selinux_user> <linux_user>
# 创建 Linux 用户时指定 SELinux 用户的映射

$ sudo semanage login -a -s <selinux_user> <linux_user>
# 将已有的 Linux 用户映射至指定 SELinux 用户

$ sudo semanage login -d -s <selinux_user> <linux_user>
# 删除 Linux 用户与指定 SELinux 用户的映射

### 示例 ###
$ sudo useradd seuser1
$ sudo echo "redhat" | passwd --stdin seuser1
$ sudo semanage login -a -s system_u seuser1
$ sudo semanage login -l

Login Name           SELinux User         MLS/MCS Range        Service

__default__          unconfined_u         s0-s0:c0.c1023       *
root                 unconfined_u         s0-s0:c0.c1023       *
seuser1              system_u             s0-s0:c0.c1023       *

$ sudo semanage login -d -s system_u seuser1
$ sudo semanage login -l

Login Name           SELinux User         MLS/MCS Range        Service

__default__          unconfined_u         s0-s0:c0.c1023       *
root                 unconfined_u         s0-s0:c0.c1023       *
```

- 受限 SELinux 用户示例：

  执行以下步骤分别测试不同的 SELinux 用户：

  ```
  # for i in {1..3}; do useradd seuser$i; echo "redhat" | passwd --stdin seuser$i;
  done
  # usermod -aG wheel seuser1    # 添加 seuser1 用户至 wheel 附加组中 → 测试 user_u
  # usermod -aG wheel seuser2    # 添加 seuser1 用户至 wheel 附加组中 → 测试 sysadm_u
  # semanage login -a -s user_u seuser1
  # semanage login -a -s sysadm_u seuser2
  ```

  - `user_u`：
    - 标准的非管理用户帐户

- 映射至 user_u 的 Linux 用户限制：
  - 1 无法通过 su 切换至其他用户
  - 2 无法 sudo 提权为 root 用户
  - 3 无法执行大多数 SUID 程序

```
$ ssh seuser1@servera
$ sudo lvs
sudo: PERM_SUDOERS: setresuid(-1, 1, -1): Operation not permitted
sudo: no valid sudoers sources found, quitting
sudo: setresuid() [0, 0, 0] -> [1002, -1, -1]: Operation not permitted    # sudo 提权失败
sudo: error initializing audit plugin sudoers_audit
$ su - student
Password:
su: Authentication failure    # 切换其他用户失败
```

- `sysadm_u`：
  - 该 SELinux 用户用于系统管理
  - 映射到 sysadm_u 的 Linux 用户：
    - 1 允许通过 su 切换至其他用户
    - 2 允许 sudo 提权为 root 用户
    - 3 允许执行 SUID 程序
  - 映射到 sysadm_u 的用户默认无法使用 SSH 登录，需将 **ssh_sysadm_login 布尔值**（boolean）设为 on。

```
### 测试 SSH 登录 ###
## 终端1
$ ssh seuser2@servera    # 映射至 sysadm_u 的用户默认无法使用 SSH 登录
client_loop: send disconnect: Broken pipe

## 终端2
# setsebool ssh_sysadm_login on    # 启用 ssh_sysadm_login 布尔值
# getsebool -a | grep ssh_sysadm_login
ssh_sysadm_login --> on

## 终端1
$ ssh seuser2@servera    # 登录成功
Activate the web console with: systemctl enable --now cockpit.socket

Register this system with Red Hat Insights: insights-client --register
Create an account or view all your systems at https://red.ht/insights-
dashboard

## 终端2
# id seuser2
uid=1003(seuser2) gid=1003(seuser2) groups=1003(seuser2),10(wheel)
# ausearch -i -ui 1003 -ts recent    # 搜索 seuser2 用户的限制 ssh 登录日志
----
type=PROCTITLE msg=audit(03/27/26 10:35:38.136:334) : proctitle=sshd: seuser2
[priv]
type=SYSCALL msg=audit(03/27/26 10:35:38.136:334) : arch=x86_64 syscall=write
success=no exit=EACCES(Permission denied) a0=0x8 a1=0x557d51084bd0 a2=0x2a
a3=0x0 items=0 ppid=1477 pid=1480 auid=seuser2 uid=seuser2 gid=seuser2
euid=seuser2 suid=seuser2 fsuid=seuser2 egid=seuser2 sgid=seuser2
fsgid=seuser2 tty=(none) ses=12 comm=sshd exe=/usr/sbin/sshd
subj=system_u:system_r:sshd_t:s0-s0:c0.c1023 key=(null)
type=AVC msg=audit(03/27/26 10:35:38.136:334) : avc:  denied  { dyntransition
} for  pid=1480 comm=sshd scontext=system_u:system_r:sshd_t:s0-s0:c0.c1023
tcontext=sysadm_u:sysadm_r:sysadm_t:s0-s0:c0.c1023 tclass=process permissive=0
```

```
### 测试 su 与 sudo 命令 ###
$ id
uid=1003(seuser2) gid=1003(seuser2) groups=1003(seuser2),10(wheel)
context=sysadm_u:sysadm_r:sysadm_t:s0-s0:c0.c1023
$ id -Z
sysadm_u:sysadm_r:sysadm_t:s0-s0:c0.c1023
$ su - student    # 切换成功
Password:
Last failed login: Fri Mar 27 09:45:51 EDT 2026 on pts/1
There were 3 failed login attempts since the last successful login.
$ sudo fdisk -l    # 提权成功
```

🤔 **Question:** 为何映射到 sysadm_u 的 Linux 用户可以运行 SUID 程序?

🔏 **Answer:** 映射至 sysadm_u 用户的 Linux 用户所在的源域为 sysadm_t，此类用户允许执行 SUID 程序。如 /usr/bin/passwd 具有 passwd_exec_t 文件上下文类型，而此二进制可执行文件运行时进程域可转换为 passwd_t 域，而 passwd_t 域的进程可对 shadow_t 文件上下文类型的文件（如 /etc/shadow）进行修改，最终完成用户密码更改。同理，/usr/bin/chsh 可更新用户登录 shell 类型。

```
## 终端1
$ id -Z
sysadm_u:sysadm_r:sysadm_t:s0-s0:c0.c1023    # 确认用户的源域 sysadm_t

## 终端2
$ sudo semanage login -l | grep seuser2
seuser2              sysadm_u             s0-s0:c0.c1023       *
$ sudo ls -lhZ /usr/bin/passwd    # 查看二进制可执行文件的文件上下文类型
-rwsr-xr-x. 1 root root system_u:object_r:passwd_exec_t:s0 32K Aug 10  2021
/usr/bin/passwd
$ sudo ls -lhZ /usr/bin/chsh
-rws--x--x. 1 root root system_u:object_r:chfn_exec_t:s0 24K Mar 29  2023
/usr/bin/chsh    # 查看二进制可执行文件的文件上下文类型

$ ls -lhZ /etc/passwd    # 查看文件上下文类型
-rw-r--r--. 1 root root system_u:object_r:passwd_file_t:s0 1.8K Mar 27 10:18
/etc/passwd
$ ls -lhZ /etc/shadow
----------. 1 root root system_u:object_r:shadow_t:s0 1.5K Apr  3 07:27
/etc/shadow

$ sudo sesearch -T -s sysadm_t -t passwd_exec_t    # 查看源域与目标域的 TE 转换
规则
type_transition sysadm_t passwd_exec_t:process passwd_t;
# 当 sysadm_t 域的进程执行 passwd_exec_t 类型的文件时，默认尝试将新进程转换到
passwd_t 域。

$ sudo sesearch -A --source passwd_t --target shadow_t --class file    # 等效
于 sesearch -A -s passwd_t -t shadow_t -c file
allow domain file_type:file map; [ domain_can_mmap_files ]:True
allow passwd_t shadow_t:file { append create getattr ioctl link lock map open
read relabelfrom relabelto rename setattr unlink watch watch_reads write };
$ sudo sesearch -A --source passwd_t --target passwd_file_t --class file
allow domain file_type:file map; [ domain_can_mmap_files ]:True
allow nsswitch_domain passwd_file_t:file { getattr ioctl lock map open read };
allow passwd_t passwd_file_t:file { append create link rename setattr unlink
watch watch_reads write };
```

- `staff_u`：

        - 映射到 staff_u 的 Linux 用户：
            - 1 允许 sudo 提权为 root 用户
            - 2 无法通过 su 切换至其他用户
        - staff_u 适用于需要使用 sudo 执行特定任务的普通用户，但无法通过使用 su 获得完整的 root 权限。
        - 如，Web 开发者可能需要 sudo 重新启动 httpd 服务。

- SELinux 的用户布尔值：

布尔值名称	布尔值功能
`ssh_sysadm_login`	允许映射到 sysadm_u 的 Linux 用户通过 SSH 登录
`user_exec_content`	允许 user_t 域的用户在其主目录和 /tmp 目录中执行应用程序
`sysadm_exec_content`	允许 sysadm_t 域的用户在其主目录和 /tmp 目录中执行应用程序
`staff_exec_content`	允许 staff_t 域的用户在其主目录和 /tmp 目录中执行应用程序

    - 核心功能：**控制受限用户能否执行自己可写目录中的程序（主目录和 /tmp），防止恶意软件下载到 /tmp 执行与用户运行不受信任的脚本，关闭以上布尔值用户只能运行系统目录（如 /usr/bin）中的程序，无法执行自己下载或创建的可执行文件**。

    - SELinux 布尔值与 TE 规则对应查询：

```
$ sudo sesearch -A -b <selinux_bool_name>
# 查看 SELinux 布尔值与 TE 规则的对应关系
```

    - 示例：user_u SELinux 用户与 user_exec_content 布尔值的关系

      1 user_exec_content 布尔值：on（启用）

```
## 终端1
$ sudo semanage login -l

Login Name           SELinux User         MLS/MCS Range        Service

__default__          unconfined_u         s0-s0:c0.c1023       *
root                 unconfined_u         s0-s0:c0.c1023       *
seuser1              user_u               s0                   *
seuser2              sysadm_u             s0-s0:c0.c1023       *

$ sudo setsebool -P user_exec_content on    # 永久启用此 SELinux 布尔值
$ sudo semanage boolean -l | grep user_exec_content
SELinux boolean                 State  Default Description
user_exec_content               (on   ,   on)  Allow user to exec content

## 终端2
$ ssh seuser1@servera
$ vim echo.sh
  #!/bin/bash
  echo "Hello World!"
$ chmod +x ./echo.sh
$ pwd
/home/seuser1
$ ls -lhZ echo.sh
-rwxr-xr-x. 1 seuser1 seuser1 user_u:object_r:user_home_t:s0 33 Mar 30 00:34
echo.sh    # 此文件的 SELinux 文件上下文标签为 user_home_t
$ sh ./echo.sh    # 测试结果1：脚本运行成功
Hello World!
$ ./echo.sh
Hello World!      # 测试结果2：脚本运行成功
```

2 user_exec_content 布尔值：off（关闭）

```
## 终端1
$ sudo setsebool -P user_exec_content off
$ sudo semanage boolean -l | grep user_exec_content
user_exec_content               (off  ,  off)  Allow user to exec content

## 终端2
$ sh ./echo.sh
Hello World!                        # 测试结果1：脚本运行成功
$ ./echo.sh
-sh: ./echo.sh: Permission denied    # 测试结果2：权限拒绝，运行失败。
```

🤔 **Question:** 当 user_exec_content 布尔值 off（关闭）的状态下，为什么使用 sh 命令执行脚本依然可以成功，而直接运行脚本却权限拒绝呢？

Answer:

- 1 无论当 user_exec_content 布尔值 off（关闭）或 on（启用）的状态下，echo.sh 文件的上下文类型始终为 user_home_t。
- 2 当 user_exec_content 布尔值 on 状态下，无论是使用 sh 命令执行还是直接执行脚本都可成功返回。通过 `sesearch -A -b user_exec_content` 命令查询可知 `allow user_usertype user_home_type:file { execute execute_no_trans getattr ioctl map open read }; [ user_exec_content ]:True` 与 `allow user_usertype user_tmp_t:file entrypoint; [ user_exec_content ]:True` 两条规则，分别表示在布尔值启用的情况下，可执行文件在用户家目录（上下文类型为 user_home_t）与 /tmp 目录（上下文类型为 user_tmp_t）中均可直接执行。使用 sh 命令直接执行是因为可执行文件仅作为 sh 命令的参数传入，/usr/bin/sh 的上下文类型为 bin_t，通过 `sesearch -A -s user_t -t bin_t -c file` 命令查询可知 `allow user_t bin_t:file entrypoint;` 规则，其表明 user_t 域的进程可以将 bin_t 类型的文件作为进入目标域的合法入口点（entrypoint），因此可直接通过 sh 命令运行。
- 3 当 user_exec_content 布尔值 off 状态下，sh 命令依然可执行该脚本，不受此布尔值影响（原因如上），而直接执行脚本运行失败是由于布尔值 off 的情况下拒绝上述两条规制的放行。因此，如果要在布尔值关闭且依然能运行脚本的话，可采用 sh 命令或 bash 命令调用的方式实现。

2.2 文件上下文类型与源域、目标域的关系

2.2.1 关系总览

类型	作用	示例
源域（Source Domain）	执行者的当前域	`init_t`，`user_t`，`unconfined_t`
文件上下文类型（Entrypoint Type）	被 **执行文件** 的类型，触发转换	`httpd_exec_t`，`passwd_exec_t`，`shell_exec_t`
目标域（Target Domain）	执行后进入的新域	`httpd_t`，`passwd_t`，`mozilla_t`

2.2.2 文件上下文类型作为桥梁

```
type_transition sysadm_t passwd_exec_t : process passwd_t;
#                    ↑          ↑            ↑        ↑
#                    |          |            |        └── 目标域（转换后的新进程域）
#                    |          |            └──────── 类别（process = 域转换）
#                    |          └──────────────────── 文件上下文类型（被执行的文件的类
型，触发转换）
#                    └──────────────────────────────── 源域（执行者的当前域）
```

文件类型 secure_services_exec_t 是触发条件 ———— 当源域执行该类型的文件时，自动转换到目标域。

示例（sysadm_t 域的用户执行 SUID 程序）： [sysadm_t] --(执行 passwd_exec_t)--> [passwd_t] --> [文件上下文类型: passwd_file_t + shadow_t]

2.2.3 文件上下文类型作为入口点（Entrypoint）

```
allow ext_gateway_t secure_services_exec_t : file entrypoint;
#     源域/主体类型    目标类型/客体类型
```

字段	作用
`ext_gateway_t`	**源域（Source Type）**：执行操作的进程域
`secure_services_exec_t`	**目标类型（Target Type）**：被执行的文件
`file`	目标类别
`entrypoint`	权限：指定该文件可作为进入目标域的入口

目标域必须将文件类型声明为 entrypoint，否则无法通过该文件进入此域。

示例：参考前文 user_exec_content 布尔值对用户执行可执行程序的影响

2.2.4 完整 Domain Transition 需要三条规则

规则	作用	示例
执行权限	源域能执行该文件	`allow unconfined_t secure_services_exec_t : file { execute read };`
入口点声明	文件是目标域的合法入口	`allow ext_gateway_t secure_services_exec_t : file entrypoint;`
域转换权限	源域能转换到目标域	`allow unconfined_t ext_gateway_t : process transition;`

结论：文件上下文类型是 Domain Transition 的 触发器（触发条件） 和 入口点 ———— 源域执行特定文件类型时，SELinux 检查该文件是否为目标域的合法入口，若是，则转换到目标域。

2.2.5 示例：sshd 启动完整流程

```
[kernel_t] --(执行 init_exec_t)--> [init_t]
   ↑                                  |
   |                                  ▼
   |                           (执行 initrc_exec_t)
   |                                  |
   |                                  ▼
   |                             [initrc_t]
   |                                  |
   |                           (执行 sshd_exec_t)
   |                                  |
   └──────────────────────────────────┘
                                      ▼
                                  [sshd_t]
```

步骤	源域	文件类型	目标域
1	`kernel_t`	`init_exec_t`	`init_t`
2	`init_t`	`initrc_exec_t`	`initrc_t`
3	`initrc_t`	`sshd_exec_t`	`sshd_t`

2.3 限制（confine）用户账户

- 实施用户限制常规方法：
  - 1 更新默认的 SELinux 映射，将 Linux 用户与 user_u 关联：

```
$ sudo semanage login -m -s user_u -r s0 __default__
```

- 2 将系统管理员，此用户不一定是 root，可以是提权为 root 的其他用户，映射到 sysadm_u SELinux 用户：

```
$ sudo semanage login -a -s sysadm_u operator1
$ sudo semanage login -a -s sysadm_u operator2
$ sudo semanage login -a -s sysadm_u operator3
```

- 3 （可选）将需要 sudo 权限的 Linux 用户映射到 staff_u，然后配置 sudo。

```
$ sudo semanage login -a -s staff_u developer1
$ sudo semanage login -a -s staff_u developer2
```

- 限制不同的 **用户账户**：
  - 若需限制所有 Linux 用户，通常首先需修改默认的映射为 user_u 映射。
  - 因此，SELinux 默认为将所有 Linux 用户映射到权限最小的一个 SELinux 用户。
  - 如需加强安全防护，还可阻止 user_u 中的用户在其家目录或 /tmp 目录中执行程序。
  - 可将 user_exec_content 布尔值设为 off（见如上示例）

```
$ sudo setsebool -P user_exec_content off
# 将user_exec_content布尔值设置为off
```

- 限制 **系统管理员**：
  - 若需限制系统管理员，将其 Linux 用户映射至 sysadm_u SELinux 用户。

```
$ sudo semanage login -a -s sysadm_u <sysadmin_name>
```

```
$ sudo useradd -G wheel -Z sysadm_u <sysadmin_name>
# 创建新用户时，将其映射为 sysadm_u SELinux 用户。
```

  - 将指定的系统管理员用户添加至 wheel 组中，使其具有 sudo 权限。
  - 默认情况下，映射至 sysadm_u 的用户不可通过 SSH 登录系统（由 ssh_sysadm_login 布尔值定义）

- ❗删除 Linux 用户时，务必删除其映射（2种方式）：

  ```
  ### 方式1 ###
  $ sudo userdel operator1
  $ sudo semanage login -d -s sysadm_u operator1

  ### 方式2 ###
  $ sudo userdel -Z operator1
  ```

- 限制 **staff 用户**：
  - 某些标准的 Linux 用户可能需要以 root 的身份运行特定命令。
  - 将其映射至 staff_u SELinux 用户帐户，然后配置 sudo。
  - 配置方法与 user_u、sysadm_u 类似。
- xguest_u 和 guest_u 等受限用户所受限制比 user_u 还多。

3. 参考链接

- 💗 **SELinux Project | GitHub**
- The SELinux Notebook | GitHub
- Chapter 2. SELinux Contexts | RedHat Docs
- SELinux 介绍 | 墨天轮
- 👉 SELinux MAC 强制访问控制
- 🔥 **SELinux 从入门到放弃**
- 🖊 比防火墙还厉害的工具 SELinux